

LnagChain Multiagent

LangChain Multiple Agents

"""

LangChain Multi-Agent Example: Collaborative Research Team

Multiple specialized agents working together on complex tasks.

"""

```
from langchain.agents import AgentExecutor, create_openai_functions_agent
from langchain_openai import ChatOpenAI
from langchain.tools import Tool
from langchain.prompts import ChatPromptTemplate, MessagesPlaceholder
from typing import Dict, List
import json
```

```
# Define specialized tools for different agents
```

```
def research_tool(query: str) -> str:
    """Research specialist tool"""
    return f"Research findings on: {query}"
```

```
def analysis_tool(data: str) -> str:
    """Data analysis tool"""
    return f"Analysis of: {data}"
```

```
def writing_tool(content: str) -> str:
    """Content writing tool"""
    return f"Formatted content: {content}"
```

```

# Agent 1: Research Agent
class ResearchAgent:
    def __init__(self, llm):
        self.tools = [
            Tool(
                name="Research",
                func=research_tool,
                description="Searches and gathers information on topics"
            )
        ]

        self.prompt = ChatPromptTemplate.from_messages([
            ("system", """"You are a Research Specialist. Your role is to:
1. Gather comprehensive information on given topics
2. Find relevant sources and data
3. Provide well-organized research summaries
Focus on accuracy and thoroughness."""),
            ("human", "{input}"),
            MessagesPlaceholder(variable_name="agent_scratchpad")
        ])

        self.agent = create_openai_functions_agent(llm, self.tools, self.prompt)
        self.executor = AgentExecutor(agent=self.agent, tools=self.tools,
verbose=True)

    def run(self, task: str) -> str:
        return self.executor.invoke({"input": task})["output"]

```

```

# Agent 2: Analysis Agent
class AnalysisAgent:
    def __init__(self, llm):
        self.tools = [
            Tool(
                name="Analyze",
                func=analysis_tool,
                description="Analyzes data and extracts insights"
            )
        ]

        self.prompt = ChatPromptTemplate.from_messages([
            ("system", """"You are a Data Analysis Specialist. Your role is to:
1. Analyze research findings and data
2. Identify patterns and trends
3. Draw meaningful conclusions
Provide clear, actionable insights."""),
            ("human", "{input}"),
            MessagesPlaceholder(variable_name="agent_scratchpad")
        ])

        self.agent = create_openai_functions_agent(llm, self.tools, self.prompt)
        self.executor = AgentExecutor(agent=self.agent, tools=self.tools,
verbose=True)

    def run(self, task: str) -> str:
        return self.executor.invoke({"input": task})["output"]

```

```

# Agent 3: Writing Agent
class WritingAgent:
    def __init__(self, llm):
        self.tools = [
            Tool(
                name="Write",
                func=writing_tool,
                description="Creates well-structured written content"
            )
        ]

        self.prompt = ChatPromptTemplate.from_messages([
            ("system", """"You are a Content Writing Specialist. Your role is to:
1. Create clear, engaging written content
2. Structure information logically
3. Ensure professional tone and formatting
Produce high-quality, readable output."""),
            ("human", "{input}"),
            MessagesPlaceholder(variable_name="agent_scratchpad")
        ])

        self.agent = create_openai_functions_agent(llm, self.tools, self.prompt)
        self.executor = AgentExecutor(agent=self.agent, tools=self.tools,
verbose=True)

    def run(self, task: str) -> str:
        return self.executor.invoke({"input": task})["output"]

```

```

# Orchestrator to coordinate agents
class TeamOrchestrator:
    def __init__(self, api_key: str):
        self.llm = ChatOpenAI(model="gpt-4", temperature=0.7, api_key=api_key)
        self.researcher = ResearchAgent(self.llm)
        self.analyzer = AnalysisAgent(self.llm)
        self.writer = WritingAgent(self.llm)

    def execute_workflow(self, topic: str) -> Dict[str, str]:
        """Execute a multi-agent workflow"""

        print(f"\n{'='*60}")
        print(f"Starting Multi-Agent Workflow for: {topic}")
        print(f"{'='*60}\n")

        # Step 1: Research Phase
        print("Phase 1: Research")
        print("-" * 60)
        research_output = self.researcher.run(
            f"Research the topic: {topic}. Provide comprehensive findings."
        )
        print(f"Research Output: {research_output}\n")

        # Step 2: Analysis Phase
        print("Phase 2: Analysis")
        print("-" * 60)
        analysis_output = self.analyzer.run(
            f"Analyze this research: {research_output}. Extract key insights."

```

```

    )
    print(f"Analysis Output: {analysis_output}\n")

    # Step 3: Writing Phase
    print("Phase 3: Writing")
    print("-" * 60)
    final_output = self.writer.run(
        f"Write a comprehensive report based on:\nResearch:
{research_output}\nAnalysis: {analysis_output}"
    )
    print(f"Final Output: {final_output}\n")

    return {
        "research": research_output,
        "analysis": analysis_output,
        "report": final_output
    }

# Example usage
if __name__ == "__main__":
    orchestrator = TeamOrchestrator(api_key="your-api-key-here")

    # Execute multi-agent workflow
    results = orchestrator.execute_workflow(
        "The impact of AI agents on business productivity"
    )

    # Print summary

```

```
print("\n" + "="*60)
print("WORKFLOW COMPLETE - SUMMARY")
print("="*60)
for phase, output in results.items():
    print(f"\n{phase.upper()}:")
    print(output[:200] + "..." if len(output) > 200 else output)
```